Private IP Address Inference in NAT Networks via Off-Path TCP Control-Plane Attack

Suraj Sharma
suraj.sharma_asp26@ashoka.edu.in
Ashoka University
Sonipat, Haryana, India

Adityavir Singh
adityavir.singh_phd22@ashoka.edu.in
Ashoka University
Sonipat, Haryana, India

Mahabir Prasad Jhanwar
mahavir.jhawar@ashoka.edu.in
Ashoka University
Sonipat, Haryana, India

Abstract

Recent work at NDSS 2024 demonstrated that widely deployed NAT behaviors in Wi-Fi routers – including port preservation, insufficient reverse path validation, and the absence of TCP window tracking enable off-path TCP hijacking attacks in NATed wireless networks. These attacks exploit design weaknesses in NAT gateway routers to detect whether *some* internal client behind the NAT maintains an active TCP connection with a target server and, upon detection, to disrupt or manipulate that connection. In this paper, we show that these behaviors have significantly broader privacy implications. We demonstrate that an off-path attacker can not only hijack active TCP connections but also accurately infer the *private IP addresses* of individual clients behind a NAT that are engaged in TCP communication with a target server. Our attack operates under the same realistic assumptions as prior work, yet leverages previously unexplored behaviors in NAT state management and TCP control-plane interactions to reconstruct the full client-side connection tuple. We evaluate our attack both in a controlled laboratory testbed and in a real-world Wi-Fi network. For SSH connections, our method reliably identifies the private IP addresses of connected clients and enables forcible termination of their TCP sessions. For HTTPS connections, although the attacker successfully terminates the underlying TCP connection, modern browsers rapidly re-establish a new connection using new ephemeral ports; nevertheless, our attack reveals the private IP addresses of the originating clients, exposing a persistent privacy leakage. Our findings demonstrate that off-path TCP hijacking attacks in NATed Wi-Fi networks pose a serious and previously unrecognized threat to client privacy, extending well beyond connection disruption to enable deanonymization of internal hosts.

Keywords

TCP Hijacking, NAT, Off-Path Attack, Network Security, Client Deanonymization

1 Introduction

Wireless local area networks have become indispensable to modern connectivity, providing flexible, cable-free access to the Internet in homes, enterprises, and public spaces. Their ease of deployment, support for mobility, and steadily increasing throughput have enabled a wide spectrum of applications, ranging from routine web browsing and video streaming to latency-sensitive communication, industrial automation, and large-scale IoT deployments. At the same time, this flexibility and ubiquity make Wi-Fi networks an attractive target for attackers.

A rich body of prior work has documented a manifold of attacks against wireless networks, including ARP poisoning [18], eavesdropping [27], rogue access point deployment [1], injection of forged wireless frames exploiting weaknesses in WPA2 implementations [32, 39], and cryptographic attacks against legacy and transitional Wi-Fi security protocols [31, 40, 41]. In response, substantial progress has been made in Wi-Fi security, evolving from WEP to WPA3 and incorporating additional defenses such as access point isolation [38] and rogue AP detection [37, 41]. As a result, many classic attacks – particularly those requiring on-path or link layer access have become significantly harder to execute in practice.

As the most widely deployed transport protocol, TCP underpins a vast majority of today's internet applications. TCP is inherently a stateful protocol; endpoints maintain per-connection state, including sequence numbers, acknowledgment numbers, and port information to ensure reliable and ordered delivery of data. This reliance on shared connection state has long made TCP an attractive target for adversarial manipulation. Against this backdrop, TCP hijacking has emerged as a well-studied attack primitive [3–8, 14, 36]. In its classical formulation, an off-path attacker (one that cannot directly observe the traffic exchanged between the endpoints) is assumed to know the IP addresses of both endpoints. The attacker's objective is to infer connection-specific parameters, such as the client's ephemeral source port and the correct sequence and acknowledgment numbers. Armed with this information, the attacker can inject forged TCP segments to terminate or otherwise manipulate the victim connection. Decades of research and successive rounds of protocol hardening have substantially raised the bar for executing such attacks [14, 22, 36].

A defining feature of modern Wi-Fi deployments is their pervasive use of Network Address Translation (NAT) gateways. NAT was originally introduced to mitigate IPv4 address exhaustion by allowing multiple private hosts, each assigned a private IP address, to share a single, globally routable public IP address. NAT is now widely deployed across Wi-Fi networks, cellular infrastructures (4G/5G), cloud platforms, and IoT ecosystems. In addition to conserving IP address space, NAT is often perceived as providing a degree of security by concealing internal network structure and private IP addresses from external observers [16, 21, 33]. Recent works have shown that this perception is overly optimistic [12, 43]. In NAT-enabled Wi-Fi networks, the TCP hijacking problem assumes a fundamentally different form: an off-path attacker knows only the public IP address of the NAT gateway and the IP address of a target server. Operating under the assumption that one or more internal hosts have established active TCP connections to the target server, the attacker attempts to hijack such connections without knowing which specific internal host is involved. Under this restricted view, Yang et al. [43] showed that several widely deployed NAT behaviors – including port preservation, insufficient reverse path validation, and the lack of TCP window tracking, can

be exploited by an off-path attacker within the LAN to inject forged TCP segments to disrupt or hijack active TCP connections.

In this work, we show that the implications of these NAT behaviors are significantly broader. We demonstrate that an off-path attacker in a NAT-enabled wireless network can, in addition to TCP connection hijacking, *deanonymize* internal hosts by uncovering the private IP addresses of the clients that maintain those connections. This capability represents a substantial privacy leakage. NAT, which is explicitly designed to hide internal host identities, fails to provide this guarantee in practice. By enabling an attacker to determine whether a specific client is communicating with a given external server and to recover the client's private IP address, our attack facilitates fine-grained profiling of users and their communication patterns within a Wi-Fi network.

Our attack builds on the same underlying NAT behaviors identified by Yang et al. [43], but leverages them in a new way. Specifically, we show that the side channel arising from widely deployed NAT behaviors, in combination with standard TCP mechanisms, can be exploited to reconstruct complete client-side TCP connection tuples including exact sequence and acknowledgment numbers of the TCP connection behind the NAT. After reconstructing these tuples, an off-path attacker can inject protocol-compliant control segments that forcibly terminate targeted TCP connections. We validate the practicality of our attack through experiments conducted both in a controlled laboratory setting and in a real-world Wi-Fi network, targeting SSH and HTTPS connections.

Contributions. In summary, this paper makes the following contributions:

- We show that existing TCP hijacking techniques in NATed Wi-Fi networks can be extended to *deanonymize* internal hosts engaged in active TCP connections with a target server, revealing a previously unrecognized privacy risk.
- We demonstrate the feasibility and impact of this attack through real-world experiments against SSH and HTTPS traffic, showing both client identification and connection termination.
- We release a proof-of-concept implementation to enable reproducibility and further research. https://anonymous.4open.science/r/private-ip-address-inference-in-nat-5D9B

2 Preliminaries

2.1 TCP

TCP is a stateful transport layer protocol [28]. Each endpoint maintains a comprehensive per-connection state, implemented as a Transmission Control Block (TCB) [10, 28]. Upon receiving a TCP segment, an endpoint first demultiplexes it to the correct TCB among potentially many active connections. This demultiplexing is performed using the 4-tuple:

$$(\texttt{Src IP}, \texttt{Src Port}, \texttt{Dest IP}, \texttt{Dest Port})$$

which uniquely identifies a TCP connection. Because TCP is stateful, it defines explicit mechanisms to *establish*, *maintain*, and *terminate* connection state – namely, the three-way handshake, data transfer, and connection termination protocols. Although this state is maintained locally at each endpoint, it evolves coherently across peers through validated TCP segment exchanges. TCP segments carry application-layer payload, and their headers encode the information necessary to drive these state transitions.

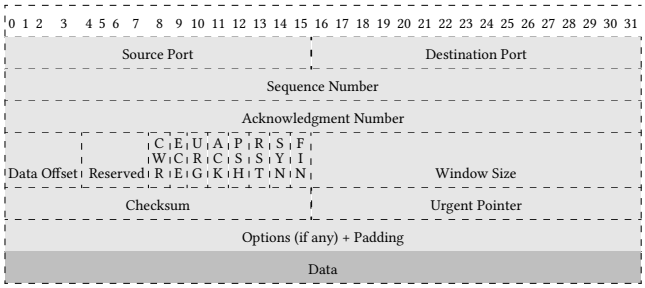


Figure 1: TCP segment format with explicit bit positions and embedded control flags.

Figure 1 shows the TCP segment structure. Operationally, three classes of TCP header fields are central to state management. First, the 4-tuple identifies the relevant connection. Second, the control flags – most notably `SYN`, `ACK`, `PSH`, `FIN`, and `RST` indicate whether a segment participates in connection establishment, steady-state communication, or termination. Third, the sequence and acknowledgment number fields implement TCP's byte-level numbering discipline for reliable and ordered delivery.

2.1.1 Segment Notation. We introduce a compact notation to describe the TCP segments crafted, forwarded, or observed during the attack. A TCP segment is represented as:

$$\mathsf{Type}^{\mathsf{FLAG}}_{\mathsf{srcIP}\to\mathsf{dstIP}}\left(\begin{array}{c}\overline{\mathsf{srcPr}\colon \mathsf{s}} \\ \overline{\mathsf{seqNo}\colon \mathsf{x}}\end{array} + \begin{array}{c}\overline{\mathsf{desPr}\colon \mathsf{d}} \\ \overline{\mathsf{ackNo}\colon \mathsf{a}}\end{array}\right),$$

where srcIP and dstIP denote the source and destination IP addresses, respectively; FLAG specifies the TCP control flags carried by the segment; s and d denote the TCP source and destination port numbers; and x and a denote the TCP sequence number and acknowledgment number, respectively. When a field is not applicable (e.g., ackNo in an initial SYN), it is denoted by "–". When random values are required (e.g., for initial TCP sequence numbers), we write $z \xleftarrow{\$} \{0,1\}^{32}$ to denote uniform sampling from the 32-bit sequence number space.

When Type = Legit, it denotes a segment whose source IP address is the actual IP address of the sender. When Type = Spoof, it denotes a segment whose source IP address is spoofed by an attacker and does not correspond to the true sender.

2.1.2 TCP state. The TCB maintains both the identifying 4-tuple and the evolving protocol state. All incoming segments are validated against the corresponding TCB before any state update is performed. We model the full TCP state at an endpoint as the tuple:

$$S = (C, S_s, S_r, S_c, S_t, S_p).$$

Here, the substate C records the current phase of the connection (e.g., establishment, data transfer, or termination), where

$C \in \{$`LISTEN`, `SYN-SENT`, `SYN-RECEIVED`, `ESTABLISHED`, `FIN-WAIT-1`, `FIN-WAIT-2`, `CLOSE-WAIT`, `CLOSING`, `LAST-ACK`, `TIME-WAIT`, `CLOSED`$\}$[10]

2.1.3 Send-sequence substate S_s. It tracks the sequence numbers of data bytes transmitted to the peer and has the structure:

S_s = (ISS, SND.UNA, SND.NXT, SND.WND)

ISS	Initial Sequence Number
SND.UNA	Sequence number of the first unacknowledged byte
SND.NXT	Next sequence number to be sent
SND.WND	Advertised send window

Incoming segments' acknowledgment numbers are matched against S_s to determine the delivery status of transmitted bytes.

2.1.4 Receive-sequence substate S_r. It tracks the sequence numbers of data bytes expected from the peer and has the structure:

S_r = (IRS, RCV.NXT, RCV.WND)

IRS	Initial Receive Sequence Number
RCV.NXT	Next expected sequence number
RCV.WND	Advertised receive window

Incoming segments' sequence numbers are matched against S_r to enforce in-order and window-constrained delivery.

2.1.5 S_c, S_t, and S_p. These remaining substates capture congestion-control variables, timer state, and negotiated connection parameters, respectively. Moving forward, these components are not required for our analysis. Hence, we restrict the TCP state to (C, S_s, S_r).

2.1.6 TCP Connection Establishment. It is a procedure (named as three-way handshake) TCP employs to establish the connection state. Figure 2 illustrates the three-way handshake together with the corresponding state evolution at the client and server.

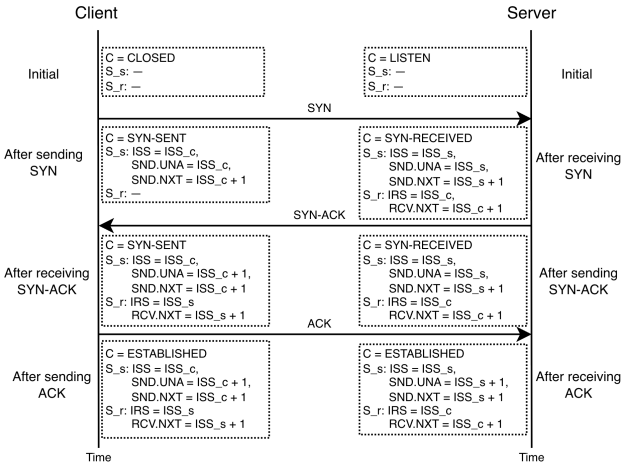


Figure 2: TCP three-way handshake showing state evolution.

2.1.7 TCP Connection Termination. A graceful TCP connection termination is typically a four-way exchange of FIN/ACK segments (sometimes observed as three when FIN and ACK are combined) [10]. One endpoint sends FIN and the peer replies with ACK; later, the peer sends its own FIN, which is acknowledged in return. Each FIN consumes one sequence number and the connection reaches CLOSED only after both directions are acknowledged.

2.2 TCP Segments Validation and State Update

TCP processes each incoming segment through a set of guarded predicates that determine whether the segment is consistent with the current state S. Formally, each predicate is a Boolean function:

$$G_{\text{pred}} : (S, \text{SEG}_{\text{in}}) \longrightarrow \{0, 1\}$$

evaluating a pred based on the incoming TCP segment (SEG_{in}) and using some substate of S. Let G_C, G_{S_s} and G_{S_r} be the predicates that validate the control, send-sequence and receive-sequence substates, respectively. A state transition occurs only if all the relevant predicates evaluate to 1.

2.2.1 G_C. It validates the incoming segment's control flags F = (SYN, ACK, FIN, RST, *etc.*) against the control substate C. It evaluates to 1 when the control flags are semantically meaningful with respect to the current phase of the connection [10]. For e.g., $G_c(S, \text{SEG}_{\text{in}}) = 0$ if $(S.C = \texttt{ESTABLISHED}) \wedge (\text{SEG}_{\text{in}}.F = \texttt{SYN})$ or $(S.C = \texttt{CLOSED}) \wedge (\text{SEG}_{\text{in}}.F = \texttt{FIN})$.

2.2.2 G_{S_s}. It validates the acknowledgment number in an incoming segment against S_s. RFC 9293 [10] specifies that $G_{S_s}(S, \text{SEG}_{\text{in}}) = 1$ if:

$$\texttt{SND.UNA} < \text{SEG}_{\text{in}}.\texttt{ACK} \leq \texttt{SND.NXT} \tag{1}$$

Window tracking on the send side is enforced using the invariant $\texttt{SND.NXT} - \texttt{SND.UNA} \leq \texttt{SND.WND}$. Valid acknowledgments advance SND.UNA, sliding the send window forward in the 32-bit sequence space.

2.2.3 G_{S_r}. It validates the sequence number in an incoming segment against S_r. RFC 9293 [10] specifies that $G_{S_r}(S, \text{SEG}_{\text{in}}) = 1$ if:

$$\texttt{RCV.NXT} \leq \text{SEG}_{\text{in}}.\texttt{SEQ} < \texttt{RCV.NXT} + \texttt{RCV.WND} \tag{2}$$

In-order data advances RCV.NXT, sliding the receive window forward and maintaining the acceptance region in the sequence space.

2.2.4 Segment Validation and State Transition. The state transition function δ based on the current state S and an incoming segment SEG_{in} is defined as follows:

$$\delta(S, \text{SEG}_{\text{in}}) = \begin{cases} (S', \text{SEG}_{\text{out}}) & \text{if} \quad G_C \wedge G_{S_s} \wedge G_{S_r} = 1 \\ (S, \text{SEG}_{\text{out}}) & \text{otherwise.} \end{cases} \tag{3}$$

where SEG_{out} denotes the output segment[1]. $\text{SEG}_{\text{out}} \in \{\emptyset, \texttt{ACK}, \texttt{RST}, \texttt{SYN-ACK}, \texttt{FIN-ACK}, \texttt{RST-ACK}, \texttt{ACK+DATA}, \texttt{challenge-ACK}\}$.

2.2.5 Challenge-ACK mechanism. RFC 5961 [36] introduced this mechanism to mitigate blind in-window SYN/RST and data injection attacks. Prior to RFC 5961, an attacker who could guess a valid four-tuple and an in-window sequence number could terminate a TCP connection using a spoofed RST or SYN, or inject data. RFC 5961 recommends strict validation of control and data segments.

If an attacker sends a spoofed SYN or RST segment with an in-window sequence number, the endpoint sends a challenge-ACK segment (c-ACK) containing the exact sequence and acknowledgment numbers to challenge the peer. In particular, if a TCP endpoint receives a segment with the SYN flag set while the connection is

[1]When $\text{SEG}_{in} = \emptyset$, the transition $\delta(S) = (S', \text{SEG}_{out})$ models locally triggered events (e.g., application sends, retransmissions, or timer-driven transmissions) that cause the endpoint to emit a segment.

already synchronized (i.e., in the ESTABLISHED state), it must respond with a c-ACK, regardless of the segment's sequence number. If the peer has genuinely lost the connection state, it will respond with a correctly formed RST segment.

Formally, RFC 5961 [36] specifies:

$G_C(S, \text{SEG}_{\text{in}}) = 0$ if $(S.C = \texttt{ESTABLISHED}) \wedge (\text{SEG}_{\text{in}}.\texttt{F} = \texttt{SYN} \vee \texttt{RST})$

Thus, $\delta(S, \text{SEG}_{\text{in}}) = (S, \text{SEG}_{\text{out}})$, where $\text{SEG}_{\text{out}} = \texttt{ACK(SEQ} = \texttt{SND.NXT, ACK} = \texttt{RCV.NXT, CTL} = \texttt{ACK)}$.

2.3 TCP Hijacking

Classical off-path TCP hijacking can be understood as a *state inference* problem. An off-path attacker aiming to hijack a connection between two endpoints must infer sufficient components of the connection's Transmission Control Block (TCB) – comprising 4-tuple and state S. Subsequently, the attacker can craft spoofed segments satisfying $G_C \wedge G_{S_s} \wedge G_{S_r} = 1$, thereby inducing valid state transitions of the form $\delta(S, \text{SEG}_{\text{in}}) = (S', \text{SEG}_{\text{out}})$. Successful state inference enables the attacker to terminate the connection or inject malicious data into plaintext streams. In the following, we formalize the smallest subset of the TCB required for mounting such attacks.

Minimal Hijacking State. We define the *Minimal Hijacking State*, denoted by $\Pi(S)$, as the smallest subset of the TCB an off-path attacker must infer to successfully hijack a connection between two endpoints:

$$\Pi(S) = \{\texttt{Src Port},\ S_r.\text{RCV.NXT},\ S_s.\text{SND.NXT}\}.$$

The client IP address (Src IP), server IP address (Dest IP), and server port (Dest Port) are typically known, reducing the attacker's inference problem to determining the ephemeral source port (Src Port) and the sequence-space values.

For connection termination via RST segments, RFC 5961 [36] mandates that SEG.SEQ = RCV.NXT. Thus, the knowledge of {Src Port, S_r.RCV.NXT} is sufficient to terminate a connection.

2.4 Network Address Translation

NAT enables multiple private hosts [29] to share a public IP address by rewriting the source IP address and port of outbound segments and maintaining the corresponding mappings in a translation table [16, 34, 35]. Inbound segments are forwarded only if they match an existing mapping, with the destination address and port rewritten to the appropriate internal host. For TCP, mappings are typically created upon observing an outbound SYN segment and are maintained using coarse per-flow state inferred from TCP control flags.

2.4.1 Port Allocation Strategies. NAT devices differ in how they select source ports for outbound TCP segments. Under port preservation strategy, the NAT gateway attempts to reuse the internal host's original ephemeral port whenever possible, falling back to another unused port only upon collision[2]. Other port allocation strategies include random or per-destination sequential allocation.

[2]When multiple internal hosts use the same source port to establish connections to the same external IP address and external port, this situation is referred to as a port collision.

2.5 TCP Hijacking in NAT Networks

TCP hijacking in NAT networks follows the same *state inference* objective as classical off-path hijacking, but the attacker's required knowledge change due to IP address and port translation performed at the network gateway. We formalize the smallest subset of the TCB required to hijack connections traversing the NAT networks.

Minimal Hijacking State in NAT. Consider a client within a NAT network connected to a well-known server on the internet. The *Minimal Hijacking State in NAT*, denoted by $\Pi_{\text{NAT}}(S)$, required by an off-path attacker to hijack the connection is:

$$\Pi_{\text{NAT}}(S) = \{\text{IP}_{pub},\ p_t,\ S_r.\text{RCV.NXT},\ S_s.\text{SND.NXT}\}.$$

The attacker must infer the public IP address of the NAT device (IP_{pub}) and the translated port (p_t) in order to craft spoofed segments with a valid 4-tuple, along with the current sequence-space values. Subsequently, the attacker can send these spoofed segments to either the NAT device's public IP address or the server to induce a state transition under δ and successfully hijack the connection.

In our attack, however, we go one step further: we not only hijack the victim connections, but also infer the private IP addresses of the clients maintaining those connections.

3 Threat Model of our Off-Path Private IP Inference Attack

We consider a network consisting of four entities: (i) a set of client machines behind a NAT gateway, (ii) a target server on the internet, (iii) the NAT gateway, and (iv) an off-path attacker located within the same LAN as the clients, as illustrated in Figure 3. The clients maintain active TCP connections with the target server. The attacker's objective is to *identify which internal clients are communicating with the target server*, i.e., to infer the private IP addresses of clients participating in active TCP connections. The attacker can also induce connection termination as part of the inference process.

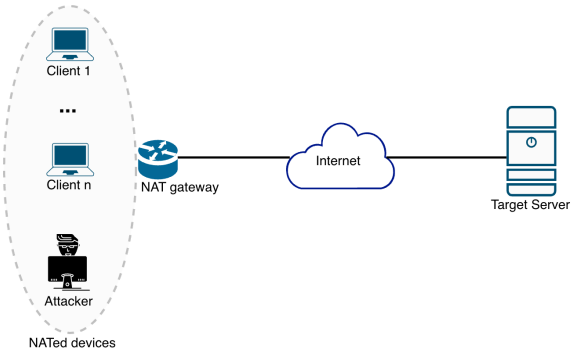


Figure 3: The Threat Model.

System Model. We consider a local area network (LAN) consisting of n client machines with private IP addresses $\text{ip}_1, \ldots, \text{ip}_n$ within the $10.0.0.0/k$ range [29]. This subnet configuration provides an address space of $2^{32-k} - 2$ assignable host addresses. We assume that all clients are running the same TCP application and communicating with a target server whose public IP address is known, say 6.6.8.8.

The gateway exposes a single public IP address to external hosts, denoted by 4.4.4.4. Each client establishes a TCP connection to the server using a locally chosen ephemeral source port; we denote these ports by $p_1, p_2, \ldots, p_n \mid p_i \neq p_j$ for all $i \neq j$. The server listens on a fixed destination port y. We refer to the set $\{p_j\}_{j=1}^{n}$ as the *victim ports* and to the corresponding set of client IP addresses $\{\mathsf{ip}_j\}_{j=1}^{n}$ as the *victim client IP addresses*.

Attacker Model. We assume an attacker located within the same LAN as the victim clients, is assigned a private IP address (say, 10.0.0.1). The attacker is *off-path* with respect to the TCP connections between the victim clients and the server – it cannot observe, intercept, or modify segments exchanged between any ip_j and 6.6.8.8. However, the attacker possesses the following capabilities:

- The attacker can transmit arbitrary TCP segments to both the remote server and the gateway router.
- The attacker can forge the source IP address of transmitted segments to impersonate any internal host ip_j or the external server.
- The attacker is assumed to know the private address space (the $/k$ prefix), which can be directly inferred from the attacker's own network configuration.

With these capabilities, the attacker aims to manipulate the translation table entries $\{(\mathsf{ip}_j, p_j)\}_{j=1}^{n}$, infer sufficient portions of live TCP state and eventually infer the private IP addresses of clients that maintain TCP connections with the target server.

4 Attack Procedure

In this section, we first state the assumptions under which our attack holds. We then describe both stages of our off-path attack. In Stage 1, the attacker infers all ephemeral ports used by clients to establish TCP connections to the target server. In Stage 2, the attacker, operating within a NATed network, infers the private IP addresses of internal clients engaged in active TCP connections with the target server and subsequently disrupts those connections.

4.1 Assumptions

Our attack relies on the following three assumptions about the NAT gateway router's behavior, all of which are consistent with widely deployed NAT implementations:

4.1.1 Absence of Reverse Path Validation. Reverse path validation verifies that a segment's source IP address is reachable via its arrival interface [2, 13]. Although Linux supports strict checking through `rp_filter` [24], many NAT routers disable or relax it for existing mappings [43]. Consequently, spoofed segments from private or external IPs are accepted on the LAN interface and can traverse the NAT.

4.1.2 Port Preservation. The NAT gateway employs a port preservation strategy, assigning the WAN-side source port to be equal to the LAN-side source port whenever no collision occurs. The n concurrent TCP connections cause the NAT gateway to create and maintain n corresponding entries in its NAT translation table. Under the port preservation assumption, these entries take the following form:

LAN Side	WAN Side
$\mathsf{ip}_1 : p_1$	$4.4.4.4 : p_1$
$\mathsf{ip}_2 : p_2$	$4.4.4.4 : p_2$
$\ldots$	$\ldots$
$\mathsf{ip}_n : p_n$	$4.4.4.4 : p_n$

4.1.3 Lack of TCP Window Tracking. The gateway router does not track TCP connection state at the level of sequence and acknowledgment numbers for RST segments. In particular, it does not validate whether incoming RST or RST-ACK segments that match an existing NAT mapping fall within the expected receive window of an established connection, and may therefore remove NAT state upon processing protocol-compliant but spoofed RST segments.

4.2 Stage 1: Inferring Victim Ports $\{p_j\}_{j=1}^{n}$

Before proceeding to infer the ephemeral ports used by the victim clients to establish TCP connections, the attacker must first determine the public IP address of the NAT device. This can be achieved in many different ways [15, 20, 43].

Afterwards, the attacker iterates over each candidate source port $r \in \{32768, 32769, \ldots, 65535\}$ and determines whether r belongs to the set $\{p_1, \ldots, p_n\}$. This search space ($2^{15} = 32768$) corresponds to the ephemeral (local) source port range typically used by TCP implementations for outgoing connections[3]. By exhaustively testing this range, the attacker eventually recovers the complete set of victim ports.

For a selected port value r, the attacker sends the following legitimate SYN segment to the server:

$$\mathsf{Legit}^{\mathsf{SYN}}_{10.0.0.1\to 6.6.8.8}\left(\begin{array}{|c|c|}\hline \mathsf{srcPr}: r & \mathsf{desPr}: y \\ \hline \mathsf{seqNo}: z \xleftarrow{\$} \{0,1\}^{32} & \mathsf{ackNo}: - \\ \hline \end{array}\right)$$

Upon receiving this segment, the NAT gateway router behaves differently depending on whether r matches an existing victim port, i.e., $r \in \{p_j\}_{j=1}^{n}$ or not.

Case1: $r = p_j$, for some j in $1 \le j \le n$. Because the port is already in use, the NAT router cannot preserve it. It allocates a new external port $r' \neq r$ and inserts the NAT table entry:

$10.0.0.1 : r$	$4.4.4.4 : r'$

It then forwards the translated SYN segment:

$$\mathsf{Legit}^{\mathsf{SYN}}_{4.4.4.4\to 6.6.8.8}\left(\begin{array}{|c|c|}\hline \mathsf{srcPr}: r' & \mathsf{desPr}: y \\ \hline \mathsf{seqNo}: z & \mathsf{ackNo}: - \\ \hline \end{array}\right)$$

Case 2: $r \neq p_j$ for all $1 \le j \le n$. Since the port is unused, the NAT preserves the port and inserts the entry:

$10.0.0.1 : r$	$4.4.4.4 : r$

It then forwards:

$$\mathsf{Legit}^{\mathsf{SYN}}_{4.4.4.4\to 6.6.8.8}\left(\begin{array}{|c|c|}\hline \mathsf{srcPr}: r & \mathsf{desPr}: y \\ \hline \mathsf{seqNo}: z & \mathsf{ackNo}: - \\ \hline \end{array}\right)$$

To avoid overwhelming the server or triggering NIDS alerts [26], the attacker prevents these forwarded segments from reaching the server. This is achieved by setting a very low TTL value (2

[3] In practice, ephemeral ports for a TCP connection typically range from 32768-61000 for Linux systems and 49152-65535 for Windows systems [12].

in our experiments), ensuring the segments are dropped by some intermediate router.

Next, the attacker sends the following spoofed SYN-ACK segment to the public IP address of the NAT gateway:

$$\text{Spoof}^{\text{SYN-ACK}}_{6.6.8.8 \to 4.4.4.4}\left(\begin{array}{|c|c|}\hline \text{srcPr: } y & \text{desPr: } r \\ \hline \text{seqNo: } u \xleftarrow{\$} \{0,1\}^{32} & \text{ackNo: } z+1 \\ \hline \end{array}\right)$$

Because the gateway router lacks proper reverse path validation, it accepts this spoofed response. The router then behaves differently depending on the earlier case:

- **Case 1: ($r = p_j$):** The NAT router forwards the SYN-ACK segment to the internal host with $\text{srcIP} = \text{ip}_j$. The attacker therefore receives no response, leaking the fact that r is indeed a victim port.
- **Case 2: ($r \notin \{p_1, \ldots, p_n\}$):** The NAT router forwards the segment back to the attacker, confirming that r is not a victim port.

Through this process, the attacker can test all 2^{15} possible ports and recover the complete set $\{p_j\}_{j=1}^{n}$.

4.3 Stage 2: Inferring Victim Client IP Addresses $\{\text{ip}_j\}_{j=1}^{n}$ and Inducing Connection Termination

We assume that the attacker is given an input set of candidate IP addresses $\mathcal{T}$ such that $\{\text{ip}_j\}_{j=1}^{n} \subseteq \mathcal{T}$. In the worst-case scenario, $\mathcal{T}$ may contain up to all 2^{24} private IP addresses in a block such as 10.0.0.0/8. The attacker processes the elements of $\mathcal{T}$ one at a time. For a fixed candidate IP address $\text{ip} \in \mathcal{T}$, the attacker iterates over the set of victim ports $\{p_j\}_{j=1}^{n}$, testing each ordered pair (ip, p_j) in turn. The goal of this inner iteration is to determine whether ip coincides with one of the victim client IP addresses.

Concretely, for a given $\text{ip} \in \mathcal{T}$ and for each p_j, $1 \le j \le n$, the attacker executes the attack procedure described below to test whether $\text{ip} = \text{ip}_j$.

- If the test succeeds for some p_j, the attacker concludes that ip is a victim client IP address, records the mapping (ip_j, p_j), and immediately moves on to the next candidate IP address in $\mathcal{T}$.
- If the test fails for all $p_j \in \{p_1, \ldots, p_n\}$, then ip does not correspond to any victim client and is discarded.

By repeating this nested iteration over all $\text{ip} \in \mathcal{T}$ in the outer loop and all p_j in the inner loop, the attacker eventually recovers the complete mapping $\{(\text{ip}_j, p_j)\}_{j=1}^{n}$.

Therefore, for a fixed candidate IP address $\text{ip} \in \mathcal{T}$, the attacker sequentially selects each $p_j \in \{p_1, \ldots, p_n\}$ and initiates a test for the pair (ip, p_j) by sending the following spoofed RST-ACK segment towards the server:

$$\text{Spoof}^{\text{RST-ACK}}_{\text{ip} \to 6.6.8.8}\left(\begin{array}{|c|c|}\hline \text{srcPr: } p_j & \text{desPr: } y \\ \hline \text{seqNo: arb} & \text{ackNo: arb} \\ \hline \end{array}\right)$$

The sequence and acknowledgment numbers are arbitrary values in the range $[0, 2^{32} - 1]$. To ensure these segments never reach the server, the attacker sets a low TTL (2 in our experiments), causing the segments to be dropped by some intermediate router.

When this spoofed RST-ACK segment reaches the NAT gateway router, its behavior depends on the presence of an existing mapping. If no mapping exists for (ip, p_j), the NAT simply discards the segment. However, if $\text{ip} = \text{ip}_j$, meaning a mapping is present, the NAT removes the mapping after its CLOSE state timeout period (typically 1-10 seconds, depending on the router firmware)[43]. Since many NAT devices avoid strict TCP window tracking due to performance overheads, they evict such mappings even when the RST-ACK contains incorrect sequence and acknowledgment numbers.

Once the attacker has sent the RST-ACK segment and waited sufficiently for the NAT timeout, it sends the following legitimate SYN segment towards the server:

$$\text{Legit}^{\text{SYN}}_{10.0.0.1 \to 6.6.8.8}\left(\begin{array}{|c|c|}\hline \text{srcPr: } p_j & \text{desPr: } y \\ \hline \text{seqNo: } z \xleftarrow{\$} \{0,1\}^{32} & \text{ackNo: } - \\ \hline \end{array}\right)$$

When this SYN reaches the gateway router, the NAT device behaves based on whether the earlier spoofed RST-ACK segment caused an entry eviction.

Case 1: $\text{ip} \neq \text{ip}_j$ (No mapping was evicted). Since Stage 1 confirmed the existence of a WAN-side entry $(4.4.4.4, p_j)$, the new SYN segment causes a WAN-side port collision under port preservation. Consequently, the NAT selects a fresh, unused WAN-side port p'_j and creates the entry:

$10.0.0.1 : p_j$	$4.4.4.4 : p'_j$

The NAT then sends the translated segment:

$$\text{Legit}^{\text{SYN}}_{4.4.4.4 \to 6.6.8.8}\left(\begin{array}{|c|c|}\hline \text{srcPr: } p'_j & \text{desPr: } y \\ \hline \text{seqNo: } z & \text{ackNo: } - \\ \hline \end{array}\right)$$

The server, which already maintains n active connections from 4.4.4.4 at source ports $\{p_j\}_{j=1}^{n}$, treats this SYN as a new connection request from $(4.4.4.4, p'_j)$ and replies with a SYN-ACK as shown in Figure 4a. The NAT forwards this SYN-ACK to the attacker. The arrival of this segment informs the attacker that $\text{ip} \neq \text{ip}_j$. The attacker then terminates this accidental new connection by sending a RST to the server.

Case 2: $\text{ip} = \text{ip}_j$ (Mapping was evicted earlier). Since the previous spoofed RST-ACK removed the entry:

$\text{ip} = \text{ip}_j : p_j$	$4.4.4.4 : p_j$

The SYN segment causes the NAT to create the following entry:

$10.0.0.1 : p_j$	$4.4.4.4 : p_j$

When the translated SYN arrives at the server, the server already has an established TCP connection at $(4.4.4.4 : p_j, 6.6.8.8 : y)$. In accordance with RFC 5961 [36], the server treats this SYN as an unexpected segment for an established connection and responds with a c-ACK. See Figure 4b.

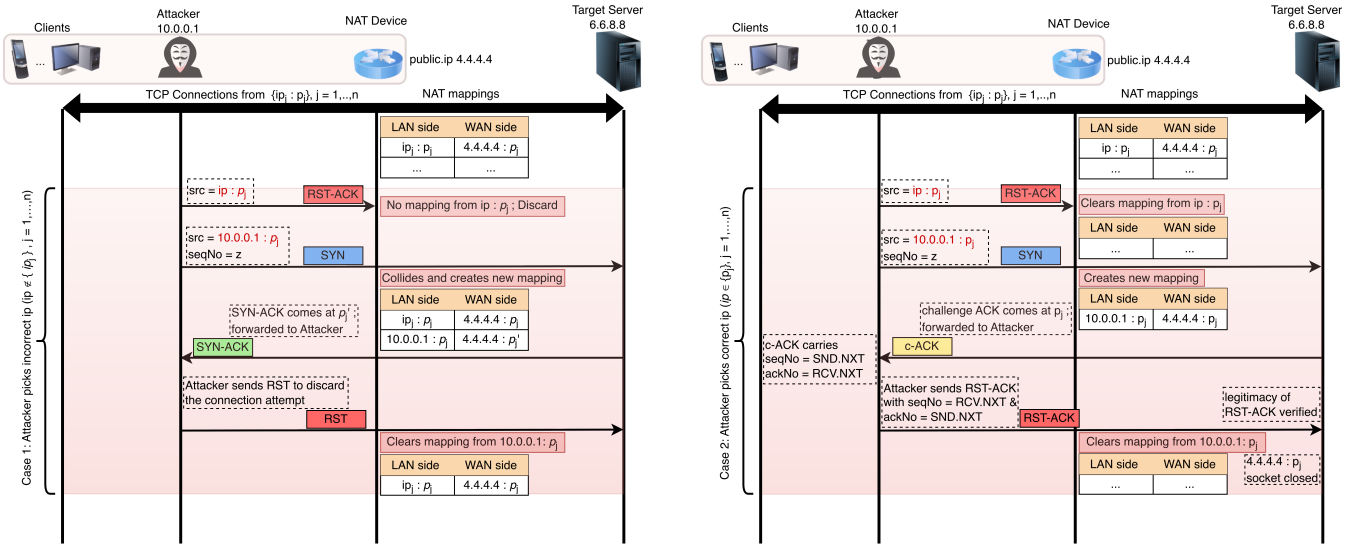
Formally, the received SYN segment violates G_C (2.2.1), i.e.,

$$G_C(S, \text{SEG}_{\text{in}}) = 0 \quad \text{when } (S.C = \texttt{ESTABLISHED}) \wedge (\text{SEG}_{\text{in}}.F = \texttt{SYN})$$

Hence, no state transition occurs and $\delta(S, \text{SEG}_{\text{in}}) = (S, \text{SEG}_{\text{out}})$, where SEG_{out} is a c-ACK segment:

$$\text{Legit}^{\text{c-ACK}}_{6.6.8.8 \to 4.4.4.4}\left(\begin{array}{|c|c|}\hline \text{srcPr: } y & \text{desPr: } p_j \\ \hline \text{seqNo: SND.NXT} & \text{ackNo: RCV.NXT} \\ \hline \end{array}\right)$$

The c-ACK carries the server's current SND.NXT and RCV.NXT values, i.e., the next sequence number the server is prepared to send



(a) Attacker picks incorrect source IP for crafting RST-ACK. **(b) Attacker picks correct source IP for crafting RST-ACK.**

Figure 4: Identifying client IP addresses connected to the target server.

and the next sequence number it expects to receive, as stored in its transmission control block for that connection. These values define the server's authoritative send/receive state and are used to validate subsequent segments on the flow.

Upon receiving the c-ACK, the NAT router forwards the translated segment to the attacker:

$$\text{Legit}^{\text{c-ACK}}_{6.6.8.8 \to 10.0.0.1}\left(\begin{array}{|c|c|}\hline \text{srcPr: } y & \text{desPr: } p_j \\ \text{seqNo: SND.NXT} & \text{ackNo: RCV.NXT} \\ \hline \end{array}\right)$$

This c-ACK confirms the attacker that $(\text{ip}, p_j) = (\text{ip}_j, p_j)$, thereby identifying one element of the true mapping $\{(\text{ip}_j, p_j)\}_{j=1}^{n}$. In addition, the attacker obtains (RCV.NXT, SND.NXT), which together with the valid 4-tuple constitutes the Minimal Hijacking State $\Pi(S)$, as defined in subsection 2.3.

The attacker then proceeds to terminate the victim's (ip_j) TCP connection by sending the following RST segment, using the server-consistent sequence and acknowledgment numbers obtained from the c-ACK segment:

$$\text{Legit}^{\text{RST}}_{10.0.0.1 \to 6.6.8.8}\left(\begin{array}{|c|c|}\hline \text{srcPr: } p_j & \text{desPr: } y \\ \text{seqNo: RCV.NXT} & \text{ackNo: SND.NXT} \\ \hline \end{array}\right)$$

This RST segment contains sequence number RCV.NXT and acknowledgment number SND.NXT, enabling successful connection termination.

5 Evaluation

We present an end-to-end evaluation measuring the time required to infer private IP addresses of clients that maintain active TCP connections to a target server. We evaluate two case studies (SSH and HTTP/HTTPS) to demonstrate denial-of-service and client-deanonymization impact.

An SSH trial is successful if the attacker infers the client's private IP and terminates the SSH session, resulting in a denial-of-service (DoS) condition for the client. For HTTP/HTTPS, success is a reliable client-IP identification. Although c-ACK reveals exact sequence and acknowledgment numbers enabling termination, browsers typically re-open connections on new ephemeral ports; thus our HTTP/HTTPS tests emphasize deanonymization rather than persistent disruption.

Ethical considerations. Experiments were run responsibly. Controlled tests used an isolated lab; real-world tests ran only with prior authorization from network administrators, using an internet-facing server and clients under our control.

5.1 Controlled Evaluation

In this section, we evaluate the attack in a controlled laboratory environment, analyze the time cost, and assess its impact on SSH and HTTP connections.

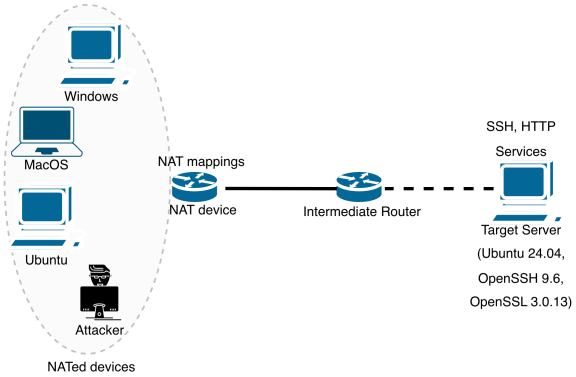


Figure 5: Experimental Setup.

5.1.1 Experimental Setup. The experimental setup for our attack in controlled environment involved four types of devices – a target server, an attacker, victim clients, and a NAT-enabled gateway router – as shown in Figure 5.

NAT router: We evaluated four consumer-grade Wi-Fi routers from different vendors: `D-Link`, `GL.iNet`, `Linksys`, and `TP-Link` (see Table 2 for details).

Server: We configured an Ubuntu 24.04 machine (kernel 6.14.0) running OpenSSH 9.6 and OpenSSL 3.0.13 that provided SSH and HTTP services to connecting clients.

Clients: Five victim clients with heterogeneous operating systems – Ubuntu 24.04.2, MacOS, Android 16, and Windows 10/11 were deployed in the NATed LAN. Our attack does not rely on operating system behavior.

Attacker: The attacker machine (MacOS Tahoe 26.3) crafts spoofed TCP segments to infer source ports of victim connections, evict NAT mappings, and ultimately determine the private IP addresses of victim clients.

5.1.2 Routers' behavior. `Linksys E5600`, `D-Link M30`, and `GL.iNet A1300` use port preservation and do not perform TCP window tracking for `RST` segments; any `RST` matching a mapping evicts it irrespective of sequence and acknowledgment numbers. None of them enforce reverse path validation as well.

`TP-Link Archer C6` behaves similarly but enables loose reverse path validation [2], blocking spoofed `SYN-ACK`s from the LAN and thwarting direct port inference. To bypass this, we use a coordinated two-attacker method: an internal attacker sends low-`TTL` `SYN`s to create NAT mappings without reaching the server (preventing NIDS triggers [26]), while an external attacker sends spoofed `SYN-ACK`s impersonating the server. Differences in response delivery reveal active external ports. This approach reduces assumptions to (i) port preservation and (ii) no TCP window tracking for `RST`, but requires an external network that disables reverse path validation [25]. Router behavior is summarized in Table 2.

The `CLOSE` state timeout value of the NAT also impacts the effectiveness of our attack. Shorter `CLOSE` timeouts reduce the overall time required to infer client IP addresses and terminate the TCP connection. In our experiments, the `TP-Link Archer C6` employed a `CLOSE` timeout of 1 s, whereas the `Linksys E5600`, `D-Link M30`, and `GL.iNet A1300` used a `CLOSE` timeout of 10 s.

5.1.3 Attack Success. Using the methodology as illustrated in subsection 4.2, together with the modification introduced in subsubsection 5.1.2, we successfully inferred all ephemeral source ports of active TCP connections on both evaluated routers, with no false positives.

Once the attacker learns the source ports used by victim client(s), it proceeds to infer their private IP address(es), as discussed in subsection 4.3. Results for /28 LANs (up to 14 hosts), including SSH and HTTP evaluations and the resulting DoS impact on SSH sessions, are summarized in Table 1.

5.1.4 Time Cost Analysis. The time required to infer a client's private IP address consists of two components. The first, denoted by T_1, corresponds to the waiting period for the NAT `CLOSE` timeout after the attacker sends a spoofed `RST-ACK` segment. The second, denoted by T_2, corresponds to the waiting time after sending a `SYN` probe and observing the server's response, which primarily depends on the network latency between the attacker and the server. Repeating this procedure for all inferred source ports and candidate IP addresses in a subnet containing n hosts and m inferred source ports yields a worst-case time cost of

$$(n-1) \cdot m \cdot (T_1 + T_2)\ seconds.$$

In our /28 subnet experiments ($n = 14$, $m = 5$), the worst-case inference time was $13 \times 5 \times (1+1) = 130$ s for the `TP-Link Archer C6` router and $13 \times 5 \times (10+1) = 715$ s for the `Linksys E5600` router.

This process can be optimized by parallelizing per-port tests for a fixed candidate IP address. The attacker sends spoofed `RST-ACK` segments simultaneously across all inferred source ports and then sends `SYN` probes using the same ports. The attacker then observes the server's response – whether it is a `c-ACK` or a `SYN-ACK`. This optimization removes the multiplicative dependence on the number of ports and yields a worst-case time cost of

$$(n-1) \cdot (T_1 + T_2)\ seconds.$$

For our /28 subnet, this corresponds to $13 \times (1+1) = 26$ s for routers with a 1 s `CLOSE` timeout (e.g., `TP-Link Archer C6`) and $13 \times (10+1) = 143$ s for routers with a 10 s timeout (e.g., `Linksys E5600`). The linear factor arises in the worst case when the target client is tested last. The total inference time can be further reduced by distributing the candidate IP space across multiple attackers, yielding an approximate $1/k$ speedup with k attackers.

Router	TO (s)	Proto.	#C	IP Inferred.	Succ.	Avg. T (s)
`Linksys E5600`	10	SSH	5	5/5	100%	65
		HTTP	5	4/5	80%	70
`TP-Link Archer C6`	1	SSH	5	5/5	100%	22
		HTTP	5	5/5	100%	24
`D-Link M30`	10	SSH	5	4/5	80%	65
		HTTP	5	4/5	80%	70
`GL.iNet GL-A1300`	10	SSH	5	5/5	100%	64
		HTTP	5	5/5	100%	68

Table 1: Controlled evaluation results. #C: number of victim clients; TO: NAT `CLOSE` state timeout.

5.1.5 Impact on SSH and HTTP sessions. Across 20 trials, the attacker correctly inferred the private IP addresses of all clients with active SSH sessions. Inference time scaled linearly with the `CLOSE` timeout, and all targeted SSH connections were terminated after NAT mapping eviction, causing a denial-of-service attack, as shown in Table 1. With five victim clients, the optimized method (as discussed in subsubsection 5.1.4) required on average 22 seconds for a 1-second `CLOSE` timeout and 65 seconds for a 10-second timeout. Owing to per-port parallelization, the inference time is independent of the number of concurrent SSH connections per client.

We evaluated the attack using a simple HTTP server with five clients behind a NAT router, each periodically fetching data at randomized 5-20 second intervals over 5 minutes. With a 1-second `CLOSE` timeout, the attacker identified the correct private IP in about 24 seconds with 100% accuracy. With a 10-second timeout, identification time increased to about 68 seconds and accuracy dropped to roughly 80%, as shown in Table 1 over 20 trials. The reduced

accuracy for longer timeouts stems from two factors: (i) client activity during the CLOSE window may refresh the NAT mapping and prevent eviction, and (ii) network latency may delay server responses, causing them to be misattributed to subsequent probe attempts, resulting in false positives.

5.2 Real-World Evaluation

5.2.1 Experimental Setup. Experiments were conducted in a Wi-Fi network with a /20 subnet, where the NAT gateway exposed a single public IP. The attacker and victims shared this LAN. For SSH, the server ran on a commercial cloud instance (Ubuntu 24.04, Linux kernel 6.14.0) with OpenSSH 9.6 and OpenSSL 3.0.13; clients used heterogeneous operating systems. For HTTPS, clients accessed a popular online banking site (www.anonymous.com[4]) over TLS 1.3.

As a /20 subnet can include up to 4096 hosts, we did not exhaustively scan the address space. Instead, we tested a selective pool of active devices, aiming to identify communicating clients, infer their private IP addresses, and disrupt their TCP sessions. As discussed in subsubsection 5.1.4, full enumeration is feasible with multiple cooperating attackers.

5.2.2 Clients running SSH sessions. We established multiple concurrent SSH sessions from victim clients to the cloud server, with the NAT creating per-flow mappings under its public IP. In all trials, the attacker correctly inferred the clients' private IPs and terminated their SSH sessions by hijacking the corresponding NAT mappings and injecting crafted TCP control segments, causing DoS attack against the victim clients.

5.2.3 Clients running HTTPS sessions. We repeated the evaluation with HTTPS, where five clients continuously accessed an online banking website over TLS 1.3. In all trials, the attacker correctly identified the internal IP addresses communicating with the server.

A representative trace is shown in Figure 6 (Appendix). During the attack, the victim receives an ICMP TTL Exceeded message (Type 11, Code 0) when the attacker spoofs the victim's IP and sends a low-TTL RST-ACK. The segment expires en route, generating ICMP while causing the NAT to move the mapping to CLOSE. The ICMP is forwarded if the entry persists within the timeout window[5].

After taking over the mapping via a SYN (confirmed by a C-ACK), the victim's segments on the hijacked port are dropped, triggering TCP retransmissions. The browser eventually times out and reconnects with a new ephemeral port, initiating a fresh TLS 1.3 handshake [30]. While application-layer recovery limits disruption duration, HTTPS remains vulnerable to off-path client identification and transient session disruption under common NAT deployments.

6 Discussion & Countermeasures

6.1 Generality of NAT Behavior

Prior measurements show that the NAT behaviors enabling our attack are widespread [12, 43]. Yang et al. [43] evaluated 67 router models from 30 vendors and 93 operational Wi-Fi networks, finding 77% of routers and roughly 80% of networks vulnerable. The exploited vulnerabilities – port preservation, weak reverse path validation, and lack of TCP window tracking for RST segments are common across consumer, enterprise, and ISP-managed devices.

These behaviors reflect performance and compatibility trade-offs: port preservation supports NAT traversal and legacy applications [44]; reverse path validation is often relaxed to tolerate asymmetric routing [2]; and TCP window tracking is omitted to reduce processing overhead and avoid application incompatibilities [12, 43]. Building on these findings, we show that the same design choices also enable off-path inference of private client IP addresses, suggesting broad applicability across NATed Wi-Fi deployments.

6.2 Attack Limitations

Frequent client-server communication. The attack may fail if a client refreshes its NAT mapping before the CLOSE timeout expires after a spoofed RST-ACK. In that case, a subsequent SYN from the attacker causes a mapping collision, leading the server to respond with SYN-ACK as for a new connection, producing a false negative. However, many routers use short timeouts (1–10 s) [43], making this condition uncommon in practice.

Large subnets. Inference time grows linearly with subnet size and the CLOSE timeout. Large address spaces (e.g., /8 or /16) increase wall-clock time but do not affect correctness, as the cost is inherently timeout-driven. As noted in subsubsection 5.1.4, multiple attackers can proportionally reduce runtime.

DHCP lease duration. DHCP assigns IP addresses for finite lease periods [9], typically around 24 hours [19]. Lease renewal limits long-term profiling based solely on IP addresses, constraining sustained deanonymization, though short-term identification remains feasible.

6.3 Countermeasures

Random Port Allocation. Routers should replace port preservation with random allocation. Since Stage 1 (4.2) depends on predictable port mappings, randomization prevents reliable source-port inference and blocks the attack at its outset.

Reverse Path Validation. Enabling strict reverse path validation per RFC 3704 [2] prevents spoofed segments received on the LAN interface with forged public source IPs, disrupting Stage 1. While this may introduce overhead or routing constraints, it substantially strengthens gateway-level spoofing resistance.

TCP Window Tracking. NAT devices should implement TCP window tracking for RST segments. Validating sequence and acknowledgment numbers before modifying mappings prevents eviction via arbitrary spoofed RST-ACK segments, thereby blocking Stage 2 (4.3) of the attack.

7 Related Work

Several off-path TCP hijacking techniques have exploited implementation and deployment subtleties to infer sequence and acknowledgment numbers and thereby terminate or inject into TCP connections [5, 7, 11, 12, 23, 42, 43]. Prior work has relied on side channels in end-host stacks and network devices to recover connection state and exploit it.

[4]The website name is anonymized for ethical reasons.
[5]ICMP packets do not refresh NAT mappings [17].

Cao et al. [5] exploited Linux's global `challenge-ACK` rate limit to recover sequence and acknowledgment numbers and detect whether two hosts were communicating, enabling off-path hijacking. Feng et al. [11] showed how forged ICMPs can downgrade Linux IPID behavior to create an IPID-based side channel for detecting connection presence and inferring sequence numbers. These studies target host-level implementation flaws; by contrast, our focus is on NAT gateway design and state-management trade-offs combined with TCP's `challenge-ACK` behavior.

NAT introduces distinct side channels and practical attack primitives [11, 42, 43]. Yang et al. [43] demonstrated that common NAT behaviors – port preservation, lack of TCP window tracking, and weak reverse path validation allow an off-path attacker within LAN, to detect and hijack internal TCP connections by manipulating the shared NAT table with spoofed segments. Their study showed such vulnerabilities are widespread across consumer and enterprise routers.

Wang et al. [42] exploit a link-layer side channel in Wi-Fi (observable encrypted frame sizes) to detect active TCP connections: by spoofing a victim IP and probing a server, an attacker monitors frame-size footprints on the shared wireless medium to confirm a connection and then recover sequence and acknowledgment numbers. That approach requires proximity to sniff link-layer traffic. In contrast, we demonstrate network-layer deanonymization: an off-path attacker can unmask internal hosts and disrupt TCP sessions solely via gateway-level interactions and crafted segments, without sniffing victim traffic or relying on physical-layer observations. This bypasses defenses aimed at link-layer side channels (frame padding, AP isolation [38], WPA2 [37, 40], WPA3).

In summary, prior work exposes both host and link-layer weaknesses enabling off-path attacks; our contribution highlights a complementary and practical network-layer vector rooted in NAT behavior and TCP control semantics, enabling client deanonymization and hijacking.

8 Conclusion

We show that common NAT behaviors in Wi-Fi routers – port preservation, weak reverse path validation, and lack of TCP window tracking for `RST` segments enable off-path attackers to hijack TCP connections and infer the private IP addresses of clients communicating with a target server. By combining NAT state manipulation with the TCP `challenge-ACK` mechanism, the attack exposes a practical privacy leak in NATed environments. Through SSH and HTTP/HTTPS case studies, we demonstrate reliable client identification and TCP session disruption, even for encrypted HTTPS traffic. This allows a low-rate attacker to infer client participation in specific connections and reveal usage patterns previously assumed hidden behind NAT. We outline three countermeasures to mitigate client deanonymization in NATed networks.

References

[1] Ali Alsahlany, Alhassan Almusawy, and Zainalabdin Alfatlawy. 2018. Risk analysis of a fake access point attack against Wi-Fi network. *International Journal of Scientific and Engineering Research* 9 (05 2018), 322–326.

[2] F. Baker and P. Savola. 2004. *Ingress Filtering for Multihomed Networks.* RFC 3704. Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc3704

[3] S. M. Bellovin. 1989. Security problems in the TCP/IP protocol suite. *SIGCOMM Comput. Commun. Rev.* 19, 2 (April 1989), 32–48. doi:10.1145/378444.378449

[4] Y. Cao, Z. Qian, Z. Wang, T. Dao, S. V. Krishnamurthy, and L. M. Marvel. 2016. Off-Path TCP Exploits: Global Rate Limit Considered Dangerous. In *Proceedings of the 25th USENIX Security Symposium (USENIX Security 2016).* USENIX Association, Austin, TX, 209–225. https://www.usenix.org/system/files/conference/usenixsecurity16/sec16_paper_cao.pdf

[5] Yue Cao, Zhiyun Qian, Zhongjie Wang, Tuan Dao, Srikanth V. Krishnamurthy, and Lisa M. Marvel. 2018. Off-Path TCP Exploits of the Challenge ACK Global Rate Limit. *IEEE/ACM Transactions on Networking* 26, 2 (2018), 765–778. doi:10.1109/TNET.2018.2797081

[6] Yue Cao, Zhongjie Wang, Zhiyun Qian, Chengyu Song, Srikanth V. Krishnamurthy, and Paul Yu. 2019. Principled Unearthing of TCP Side Channel Vulnerabilities. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS '19).* Association for Computing Machinery, New York, NY, USA, 211–224. doi:10.1145/3319535.3354203

[7] Weiteng Chen and Zhiyun Qian. 2018. Off-path TCP exploit: how wireless routers can jeopardize your secrets. In *Proceedings of the 27th USENIX Conference on Security Symposium* (Baltimore, MD, USA) *(SEC'18).* USENIX Association, USA, 1581–1598.

[8] Marco de Vivo, Gabriela O. de Vivo, Roberto Koeneke, and Germinal Isern. 1999. Internet vulnerabilities related to TCP/IP and T/TCP. *SIGCOMM Comput. Commun. Rev.* 29, 1 (Jan. 1999), 81–85. doi:10.1145/505754.505760

[9] Ralph Droms. 1997. Dynamic Host Configuration Protocol. RFC 2131. doi:10.17487/RFC2131

[10] Wesley Eddy. 2022. *Transmission Control Protocol (TCP).* RFC 9293. Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc9293

[11] Xuewei Feng, Chuanpu Fu, Qi Li, Kun Sun, and Ke Xu. 2020. Off-Path TCP Exploits of the Mixed IPID Assignment. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security* (Virtual Event, USA) *(CCS '20).* Association for Computing Machinery, New York, NY, USA, 1323–1335. doi:10.1145/3372297.3417884

[12] Xuewei Feng, Yuxiang Yang, Qi Li, Xingxiang Zhan, Kun Sun, Ziqiang Wang, Ao Wang, Ganqiu Du, and Ke Xu. 2024. ReDAN: An Empirical Study on Remote DoS Attacks against NAT Networks. In *Proceedings of the Network and Distributed System Security Symposium (NDSS).* Internet Society, San Diego, CA, USA, 14 pages. doi:10.14722/ndss.2025.230972

[13] P. Ferguson and D. Senie. 2000. *Network Ingress Filtering: Defeating Denial of Service Attacks which employ IP Source Address Spoofing.* RFC 2827. Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc2827

[14] Fernando Gont and Steven M. Bellovin. 2012. Defending against Sequence Number Attacks. doi:10.17487/RFC6528

[15] Brian J Goodchild, Yi-Ching Chiu, Rob Hansen, Haonan Lua, Matt Calder, Matthew Luckie, Wyatt Lloyd, David Choffnes, and Ethan Katz-Bassett. 2017. The record route option is an option!. In *Proceedings of the 2017 Internet Measurement Conference* (London, United Kingdom) *(IMC '17).* Association for Computing Machinery, New York, NY, USA, 311–317. doi:10.1145/3131365.3131392

[16] S. Guha, K. Biswas, B. Ford, S. Sivakumar, and P. Srisuresh. 2008. *NAT Behavioral Requirements for TCP.* RFC 5382. Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc5382.txt

[17] Saikat Guha, Bryan Ford, Senthil Sivakumar, and Pyda Srisuresh. 2009. NAT Behavioral Requirements for ICMP. RFC 5508. doi:10.17487/RFC5508

[18] Sherin Hijazi and Mohammad S. Obaidat. 2018. Address resolution protocol spoofing attacks and security approaches: A survey. *Security and Privacy* 2, 1 (Dec. 2018), 17 pages. doi:10.1002/spy2.49

[19] IBM Corporation. 2022. *IBM i 7.5: DHCP leases.* International Business Machines Corporation. https://www.ibm.com/docs/en/i/7.5.0?topic=concepts-leases Accessed: January 30, 2026.

[20] ifconfig.co. 2026. ifconfig.co. https://ifconfig.co/ Accessed: 2026-04-15.

[21] Cullen Fluffy Jennings and Francois Audet. 2007. Network Address Translation (NAT) Behavioral Requirements for Unicast UDP. RFC 4787. doi:10.17487/RFC4787

[22] Michael Larsen and Fernando Gont. 2011. Recommendations for Transport-Protocol Port Randomization. doi:10.17487/RFC6056

[23] Guancheng Li, Minghao Zhang, Jianjun Chen, Ge Dai, Pinji Chen, Huiming Liu, Yang Yu, Haixin Duan, and Zhiyun Qian. 2025. The Danger of Packet Length Leakage: Off-path TCP/IP Hijacking Attacks Against Wireless and Mobile Networks. In *2025 IEEE 10th European Symposium on Security and Privacy (EuroS&P).* IEEE, New York, USA, 807–821. doi:10.1109/EuroSP63326.2025.00051

[24] Linux Kernel Developers. 2025. rp_filter. https://sysctl-explorer.net/net/ipv4/rp_filter/

[25] Matthew J. Luckie, Robert Beverly, Ryan Koga, Ken Keys, Joshua A. Kroll, and K. C. Claffy. 2019. Network Hygiene, Incentives, and Regulation: Deployment of Source Address Validation in the Internet. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security (CCS 2019) (CCS '19).* ACM, London, United Kingdom, 465–480. doi:10.1145/3319535.3354232

[26] Varun Mahajan and Sateesh K Peddoju. 2017. Deployment of Intrusion Detection System in Cloud: A Performance-Based Study. In *2017 IEEE Trustcom/BigDataSE/ICESS.* IEEE, Piscataway, NJ, USA, 1103–1108. doi:10.1109/Trustcom/BigDataSE/ICESS.2017.359

[27] Toshihiro Ohigashi and Masakatu Morii. 2009. A practical message falsification attack on WPA. *Proc. JWIS* 54 (2009), 66.

[28] Jon Postel. 1981. *Transmission Control Protocol.* RFC 793. Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc793.txt

[29] Yakov Rekhter, Robert Moskowitz, Daniel Karrenberg, Geert Jan de Groot, and Eliot Lear. 1996. *Address Allocation for Private Internets.* RFC 1918. Internet Engineering Task Force. doi:10.17487/RFC1918

[30] Eric Rescorla. 2018. *The Transport Layer Security (TLS) Protocol Version 1.3.* RFC 8446. Internet Engineering Task Force. doi:10.17487/RFC8446

[31] Domien Schepers, Aanjhan Ranganathan, and Mathy Vanhoef. 2019. Practical Side-Channel Attacks against WPA-TKIP. In *Proceedings of the 2019 ACM Asia Conference on Computer and Communications Security* (Auckland, New Zealand) *(Asia CCS '19)*. Association for Computing Machinery, New York, NY, USA, 415–426. doi:10.1145/3321705.3329832

[32] Domien Schepers, Aanjhan Ranganathan, and Mathy Vanhoef. 2023. Framing Frames: Bypassing Wi-Fi Encryption by Manipulating Transmit Queues. In *32nd USENIX Security Symposium (USENIX Security 23)*. USENIX Association, Anaheim, CA, 53–68. https://www.usenix.org/conference/usenixsecurity23/presentation/schepers

[33] Matt Smith and Ray Hunt. 2002. Network security using NAT and NAPT. In *Proceedings 10th IEEE International Conference on Networks (ICON 2002). Towards Network Superiority (Cat. No.02EX588)*. IEEE, Piscataway, NJ, USA, 355–360. https://api.semanticscholar.org/CorpusID:31862892

[34] P. Srisuresh and K. Egevang. 2001. *Traditional IP Network Address Translator (Traditional NAT).* RFC 3022. Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc3022

[35] P. Srisuresh and M. Holdrege. 1999. *IP Network Address Translator (NAT) Terminology and Considerations.* RFC 2663. Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc2663

[36] Randall R. Stewart, Mitesh Dalal, and Anantha Ramaiah. 2010. *Improving TCP's Robustness to Blind In-Window Attacks.* RFC 5961. Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc5961

[37] Erik Tews and Martin Beck. 2009. Practical attacks against WEP and WPA. In *Proceedings of the Second ACM Conference on Wireless Network Security* (Zurich, Switzerland) *(WiSec '09)*. Association for Computing Machinery, New York, NY, USA, 79–86. doi:10.1145/1514274.1514286

[38] TP-Link. 2026. Brief Introduction of AP Isolation. https://www.tp-link.com/us/support/faq/2089/

[39] Mathy Vanhoef. 2021. Fragment and Forge: Breaking Wi-Fi Through Frame Aggregation and Fragmentation. In *30th USENIX Security Symposium (USENIX Security 21)*. USENIX Association, Berkeley, CA, USA, 161–178. https://www.usenix.org/conference/usenixsecurity21/presentation/vanhoef

[40] Mathy Vanhoef and Frank Piessens. 2017. Key Reinstallation Attacks: Forcing Nonce Reuse in WPA2. In *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security* (Dallas, Texas, USA) *(CCS '17)*. Association for Computing Machinery, New York, NY, USA, 1313–1328. doi:10.1145/3133956.3134027

[41] Mathy Vanhoef and Frank Piessens. 2018. Release the Kraken: New KRACKs in the 802.11 Standard. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security* (Toronto, Canada) *(CCS '18)*. Association for Computing Machinery, New York, NY, USA, 299–314. doi:10.1145/3243734.3243807

[42] Ziqiang Wang, Xuewei Feng, Qi Li, Kun Sun, Yuxiang Yang, Mengyuan Li, Ganqiu Du, Ke Xu, and Jianping Wu. 2025. Off-Path TCP Hijacking in Wi-Fi Networks: A Packet-Size Side Channel Attack. In *32nd Annual Network and Distributed System Security Symposium, NDSS 2025, San Diego, California, USA, February 24–28, 2025*. The Internet Society, San Diego, CA, USA. https://www.ndss-symposium.org/ndss-paper/off-path-tcp-hijacking-in-wi-fi-networks-a-packet-size-side-channel-attack/

[43] Yuxiang Yang, Xuewei Feng, Qi Li, Kun Sun, Ziqiang Wang, and Ke Xu. 2024. Exploiting Sequence Number Leakage: TCP Hijacking in NAT-Enabled Wi-Fi Networks. In *Proceedings of the NDSS Symposium 2024*. Internet Society, San Diego, CA, USA, 419–438. https://arxiv.org/abs/2404.04601

[44] Zhang Yongjun and Zhen Shiquan. 2013. NAT Hole Punching Based on Simultaneous TCP Open. In *Proceedings of the 2013 Fifth International Conference on Multimedia Information Networking and Security (MINES '13)*. IEEE Computer Society, USA, 235–238.

A Snapshot of HTTPS Connection Disruption at the Client Side

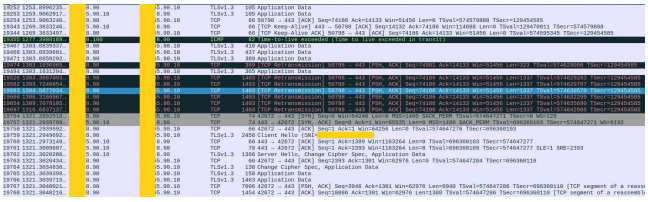


Figure 6: The impact of the attack as observed at the victim clients' end who is engaged in an HTTPS connection with a server.

B Tested Routers

Vendor	Model	PP	No WT	No RPV	Vuln.
Linksys	E5600	✓	✓	✓	✓
TP-Link	Archer C6	✓	✓	✗	✓[a]
D-Link	M30	✓	✓	✓	✓
GL.iNet	GL-A1300	✓	✓	✓	✓

Table 2: Observed NAT behaviors in evaluated consumer routers. PP: Port Preservation; WT: TCP Window Tracking; RPV: Reverse Path Validation. (✓): present; (✗): absent. (✓[a]) vulnerable with modification proposed in subsubsection 5.1.2.